



Campus: Polo Ribeiraria

Curso: 7124 - Desenvolvimento Full Stack

Disciplina: DGT2821 - Programação Back-end Com Java

Turma: 9001 **Semestre:** 2026.1

Matrícula: 2025 0324 5401

Aluno: Alexandre Vecchi

1. Título da Prática

2º Procedimento | Criação do Cadastro em Modo Texto

2. Objetivo da Prática

1. Utilizar herança e polimorfismo na definição de entidades.
2. Utilizar persistência de objetos em arquivos binários.
3. Implementar uma interface cadastral em modo texto.
4. Utilizar o controle de exceções da plataforma Java.
5. No final do projeto, o aluno terá implementado um sistema cadastral em Java, utilizando os recursos da programação orientada a objetos e a persistência em arquivos binários.

3. Todos os códigos solicitados neste roteiro de aula

1. mainapp.model – Pessoa.java

```
package model;

import java.io.Serializable;

public class Pessoa implements Serializable {

    private int id;
    private String nome;

    public Pessoa() {
    }

    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public void exibir() {
        System.out.println("Id: " + id);
        System.out.println("Nome: " + nome);
    }
}
```

2. mainapp.model – PessoaFisica.java

```
package model;

import java.io.Serializable;

public class PessoaFisica extends Pessoa implements Serializable {

    private String cpf;
    private int idade;

    public PessoaFisica() {
    }

    public PessoaFisica(int id, String nome, String cpf, int idade) {
        super(id, nome);
        this.cpf = cpf;
        this.idade = idade;
    }

    public String getCpf() {
        return cpf;
    }

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }
}
```

3. mainapp.model – PessoaFisicaRepo.java

```
package model;

import java.io.*;
import java.util.ArrayList;

public class PessoaFisicaRepo {

    private ArrayList<PessoaFisica> listaPessoas = new ArrayList<>();

    public void inserir(PessoaFisica pessoa) {
        listaPessoas.add(pessoa);
    }

    public void alterar(PessoaFisica pessoaAlterada) {
        for (int i = 0; i < listaPessoas.size(); i++) {
            if (listaPessoas.get(i).getId() == pessoaAlterada.getId()) {

```

```

        listaPessoas.set(i, pessoaAlterada);
        return;
    }
}

public void excluir(int id) {
    listaPessoas.removeIf(p -> p.getId() == id);
}

public PessoaFisica obter(int id) {
    for (PessoaFisica p : listaPessoas) {
        if (p.getId() == id) {
            return p;
        }
    }
    return null;
}

public ArrayList<PessoaFisica> obterTodos() {
    return listaPessoas;
}

public void persistir(String nomeArquivo) throws IOException {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(nomeArquivo))) {
        out.writeObject(listaPessoas);
        System.out.println("Dados de Pessoa Fisica Armazenados.");
    }
}

public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(nomeArquivo))) {
        listaPessoas = (ArrayList<PessoaFisica>) in.readObject();
        System.out.println("Dados de Pessoa Fisica Recuperados.");
    }
}
}

```

4. mainapp.model – PessoaJuridica.java

```

package model;

import java.io.Serializable;

public class PessoaJuridica extends Pessoa implements Serializable {
    private String cnpj;

    public PessoaJuridica() {
    }

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }
}

```

5. mainapp.model – PessoaJuridicaRepo.java

```
package model;

import java.io.*;
import java.util.ArrayList;

public class PessoaJuridicaRepo {

    private ArrayList<PessoaJuridica> listaPessoas = new ArrayList<>();

    public void inserir(PessoaJuridica pessoa) {
        listaPessoas.add(pessoa);
    }

    public void alterar(PessoaJuridica pessoaAlterada) {
        for (int i = 0; i < listaPessoas.size(); i++) {
            if (listaPessoas.get(i).getId() == pessoaAlterada.getId()) {
                listaPessoas.set(i, pessoaAlterada);
                return;
            }
        }
    }

    public void excluir(int id) {
        listaPessoas.removeIf(p -> p.getId() == id);
    }

    public PessoaJuridica obter(int id) {
        for (PessoaJuridica p : listaPessoas) {
            if (p.getId() == id) {
                return p;
            }
        }
        return null;
    }

    public ArrayList<PessoaJuridica> obterTodos() {
        return listaPessoas;
    }

    public void persistir(String nomeArquivo) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(nomeArquivo))) {
            out.writeObject(listaPessoas);
            System.out.println("Dados de Pessoa Juridica Armazenados.");
        }
    }

    public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(nomeArquivo))) {
            listaPessoas = (ArrayList<PessoaJuridica>) in.readObject();
            System.out.println("Dados de Pessoa Juridica Recuperados.");
        }
    }
}
```

6. mainapp.mainapp – MainApp.java

```
package mainapp;

import java.util.List;
import model.PessoaFisica;
```

```

import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;

import java.util.Scanner;

public class MainApp {

    private static final Scanner scanner = new Scanner(System.in);
    private static final PessoaFisicaRepo pessoaFisicaRepo = new PessoaFisicaRepo();
    private static final PessoaJuridicaRepo pessoaJuridicaRepo = new PessoaJuridicaRepo();
    private static final String arquivoPessoaFisica = ".fisica.bin";
    private static final String arquivoPessoaJuridica = ".juridica.bin";

    public static void main(String[] args) {

        int opcao;

        do {
            exibirMenu();
            opcao = lerInteiro("Selecione uma Opcao");

            switch (opcao) {
                case 1:
                    incluir();
                    break;
                case 2:
                    alterar();
                    break;
                case 3:
                    excluir();
                    break;
                case 4:
                    exibirPorId();
                    break;
                case 5:
                    exibirTodos();
                    break;
                case 6:
                    salvar();
                    break;
                case 7:
                    recuperar();
                    break;
                case 0:
                    System.out.println("Programa finalizado.");
                    break;
                default:
                    System.out.println("Opcao invalida.");
                    break;
            }
        } while (opcao != 0);
    }

    private static void exibirMenu() {
        System.out.println("=====");
        System.out.println("1 - Incluir Pessoa");
        System.out.println("2 - Alterar Pessoa");
        System.out.println("3 - Excluir Pessoa");
        System.out.println("4 - Buscar pelo Id");
        System.out.println("5 - Exibir Todos");
        System.out.println("6 - Persistir Dados");
        System.out.println("7 - Recuperar Dados");
    }
}

```

```

        System.out.println("0 - Finalizar Programa");
        System.out.println("=====");
    }

    private static void incluir() {
        String tipo = lerTipo();
        if (tipo.equalsIgnoreCase("F")) {
            pessoaFisicaRepo.inserir(lerPessoaFisica());
        } else {
            pessoaJuridicaRepo.inserir(lerPessoaJuridica());
        }
    }

    private static void alterar() {
        String tipo = lerTipo();
        int id = lerInteiro("ID: ");

        if (tipo.equalsIgnoreCase("F")) {
            PessoaFisica pessoaFisica = pessoaFisicaRepo.obter(id);
            if (pessoaFisica != null) {
                pessoaFisica.exibir();
                pessoaFisicaRepo.alterar(lerPessoaFisicaComId(id));
            } else {
                System.out.println("Pessoa Fisica nao encontrada.");
            }
        } else {
            PessoaJuridica pessoaJuridica = pessoaJuridicaRepo.obter(id);
            if (pessoaJuridica != null) {
                pessoaJuridica.exibir();
                pessoaJuridicaRepo.alterar(lerPessoaJuridicaComId(id));
            } else {
                System.out.println("Pessoa Juridica nao encontrada.");
            }
        }
    }

    private static void excluir() {
        String tipo = lerTipo();
        int id = lerInteiro("ID: ");

        if (tipo.equalsIgnoreCase("F")) {
            pessoaFisicaRepo.excluir(id);
        } else {
            pessoaJuridicaRepo.excluir(id);
        }
    }

    private static void exibirPorId() {
        String tipo = lerTipo();
        int id = lerInteiro("Digita o id da Pessoa:");

        if (tipo.equalsIgnoreCase("F")) {
            PessoaFisica pessoaFisica = pessoaFisicaRepo.obter(id);
            if (pessoaFisica != null) {
                pessoaFisica.exibir();
            } else {
                System.out.println("Pessoa Fisica nao encontrada.");
            }
        } else {
            PessoaJuridica pessoaJuridica = pessoaJuridicaRepo.obter(id);
            if (pessoaJuridica != null) {
                pessoaJuridica.exibir();
            } else {

```

```

        System.out.println("Pessoa Juridica nao encontrada.");
    }
}

private static void exibirTodos() {
    String tipo = lerTipo();

    if (tipo.equalsIgnoreCase("F")) {
        List<PessoaFisica> lista = pessoaFisicaRepo.obterTodos();

        if (lista == null || lista.isEmpty()) {
            System.out.println("Lista de Pessoas Fisicas Vazia!");
        } else {
            for (PessoaFisica pessoaFisica : lista) {
                if (pessoaFisica != null) {
                    pessoaFisica.exibir();
                    System.out.println();
                }
            }
        }
    } else {
        List<PessoaJuridica> lista = pessoaJuridicaRepo.obterTodos();

        if (lista == null || lista.isEmpty()) {
            System.out.println("Lista de Pessoas Juridicas Vazia!");
        } else {
            for (PessoaJuridica pessoaJuridica : lista) {
                if (pessoaJuridica != null) {
                    pessoaJuridica.exibir();
                    System.out.println();
                }
            }
        }
    }
}

private static void salvar() {
    System.out.println("Prefixo dos arquivos: ");
    String prefixo = scanner.nextLine();

    try {
        pessoaFisicaRepo.persistir(prefixo + arquivoPessoaFisica);
        pessoaJuridicaRepo.persistir(prefixo + arquivoPessoaJuridica);
        System.out.println("Dados salvos com sucesso.");
    } catch (Exception e) {
        System.out.println("Erro ao salvar dados: " + e.getMessage());
    }
}

private static void recuperar() {
    System.out.println("Prefixo dos arquivos: ");
    String prefixo = scanner.nextLine();

    try {
        pessoaFisicaRepo.recuperar(prefixo + arquivoPessoaFisica);
        pessoaJuridicaRepo.recuperar(prefixo + arquivoPessoaJuridica);
        System.out.println("Dados recuperados com sucesso.");
    } catch (Exception e) {
        System.out.println("Erro ao recuperar dados: " + e.getMessage());
    }
}

```



```
private static PessoaFisica lerPessoaFisica() {
    int id = lerInteiro("Digite o id da Pessoa: ");
    return lerPessoaFisicaComId(id);
}

private static PessoaFisica lerPessoaFisicaComId(int id) {
    System.out.println("Insira os dados...");
    System.out.println("Nome: ");
    String nome = scanner.nextLine();
    System.out.println("CPF: ");
    String cpf = scanner.nextLine();
    int idade = lerInteiro("Idade: ");

    return new PessoaFisica(id, nome, cpf, idade);
}

private static PessoaJuridica lerPessoaJuridica() {
    int id = lerInteiro("Digite o id da Pessoa: ");
    return lerPessoaJuridicaComId(id);
}

private static PessoaJuridica lerPessoaJuridicaComId(int id) {
    System.out.println("Insira os dados...");
    System.out.println("Nome: ");
    String nome = scanner.nextLine();
    System.out.println("CNPJ: ");
    String cnpj = scanner.nextLine();

    return new PessoaJuridica(id, nome, cnpj);
}

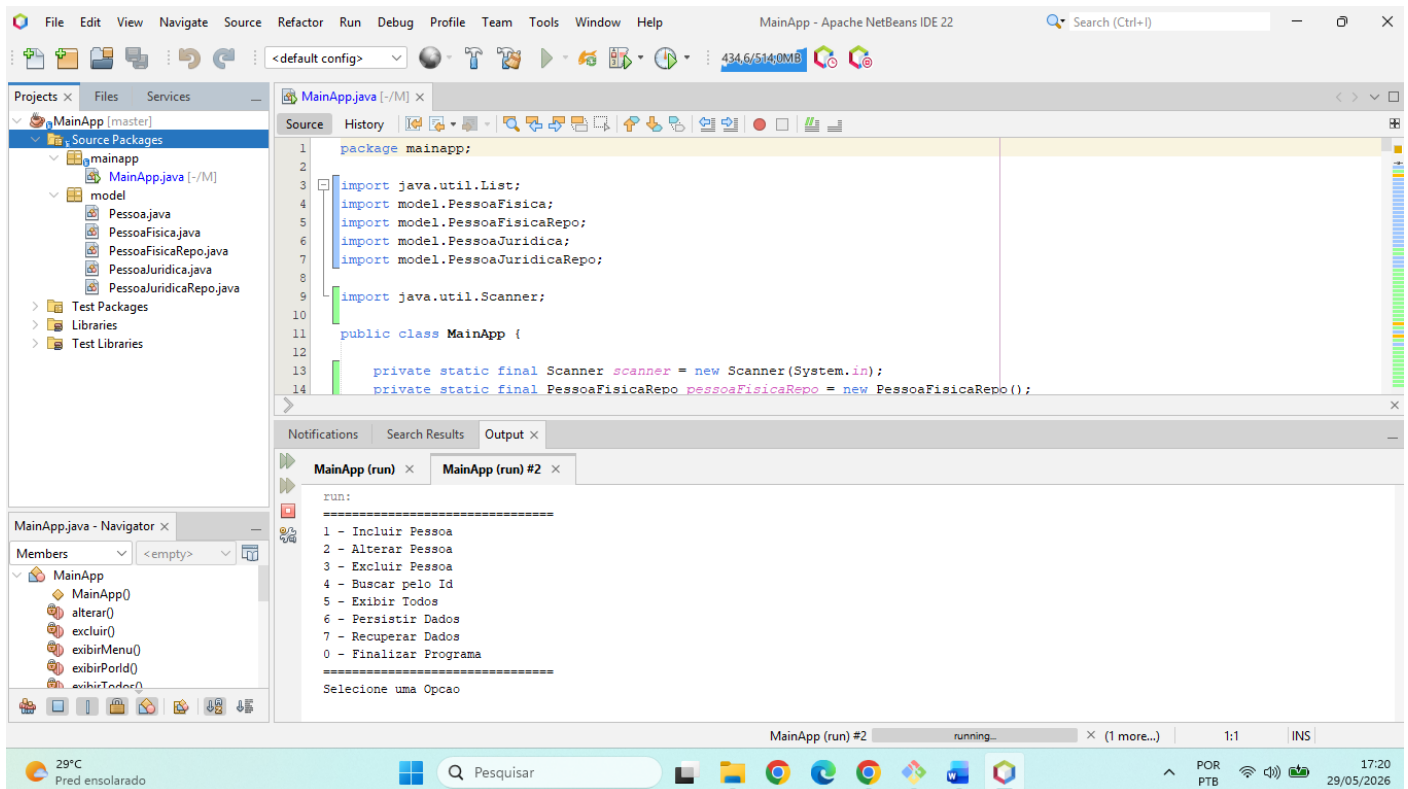
private static String lerTipo() {
    String tipo;
    do {
        System.out.println("F - Pessoa Fisica | J - Pessoa Juridica");
        tipo = scanner.nextLine();
    } while (!tipo.equalsIgnoreCase("F") && !tipo.equalsIgnoreCase("J"));

    return tipo;
}

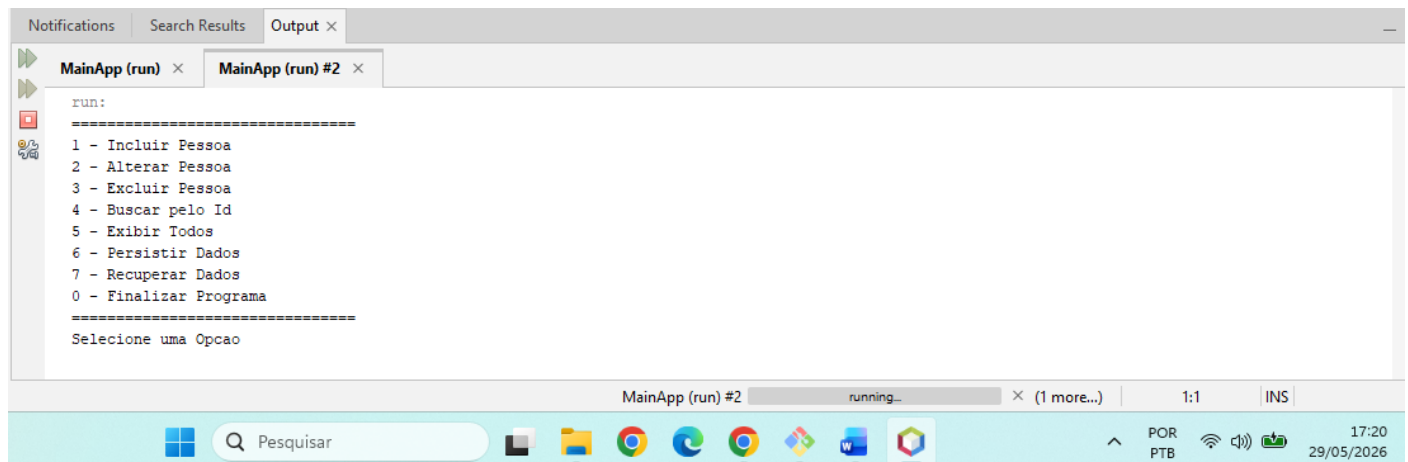
private static int lerInteiro(String mensagem) {
    System.out.println(mensagem);
    int valor = scanner.nextInt();
    scanner.nextLine();
    return valor;
}
}
```

4. Os resultados da execução dos códigos também devem ser apresentados

Executando a aplicação



Resultado da execução



5. Análise e Conclusão

a. O que são elementos estáticos e qual o motivo para o método main adotar esse modificador?

Elementos estáticos (modificador static) pertencem à **classe**, e não às instâncias (objetos) dela.

- **O que são:** São métodos ou atributos compartilhados por todos os objetos da classe. Eles podem ser chamados diretamente pelo nome da classe, sem precisar usar o comando new.
- **Motivo no método main:** O Java precisa de um ponto de partida para executar o programa. Como nenhum objeto foi criado ainda quando o programa inicia, o método main deve ser static para que a Máquina Virtual Java (JVM) consiga invocá-lo diretamente a partir da classe.

b. Para que serve a classe Scanner?

A classe Scanner serve para **ler dados de entrada** em aplicações Java.

- Ela traduz textos vindos de diferentes fontes (como o teclado do usuário ou um arquivo).
- Ela converte esses textos diretamente em tipos de dados específicos do Java, como números inteiros (nextInt()), números decimais (nextDouble()) ou linhas de texto (nextLine()).

c. Como o uso de classes de repositório impactou na organização do código?

O uso de classes de repositório (padrão *Repository*) organiza o código ao **separar a lógica de dados do restante da aplicação**.

- **Isolamento do banco:** Centraliza todas as operações de banco de dados (inserir, buscar, deletar) em um só lugar.
- **Código mais limpo:** Evita que regras de negócio fiquem misturadas com comandos SQL ou códigos de persistência.
- **Fácil manutenção:** Se o banco de dados mudar, você altera apenas a classe de repositório, sem quebrar o resto do sistema.